Lab-01: Write a JAVA Program to Display Image using JFrame

 # Source Code in JAVA:

```
 import java.awt.FlowLayout;
import javax.swing.ImageIcon;
 import javax.swing.JFrame;
 import javax.swing.JLabel;
 public class Image extends JFrame {
private ImageIcon image1;
 private JLabel label1;
 private ImageIcon image2;
 private JLabel label2;
Image(){
 setLayout(new FlowLayout());
 image1 = new ImageIcon(getClass().getResource("ice.jpg")); label1 =
new JLabel(image1);
 add(label1);
image2 = new ImageIcon(getClass().getResource("pust.png")); label2 =
new JLabel(image2);
 add(label2);
 }
 public static void main(String args[]) {
 Image gui = new Image();
gui.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
gui.setVisible(true);
gui.pack();
gui.setTitle("Image Program");
}
}
```

Output:

Dept. of Information and Communication Engineering
Pabna University of Science and Technology
ICE
Since 2010

পাবনা বিজ্ঞান ও প্রযুক্তি বিশ্ববিদ্যালয়
PABNA UNIVERSITY OF SCIENCE AND TECHNOLOGY

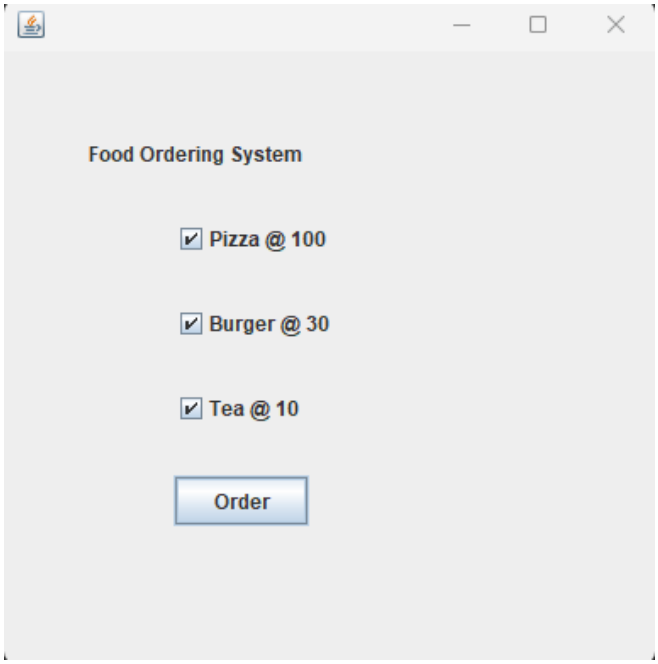
Lab-02: Write a JAVA Program for generating Restaurant Bill

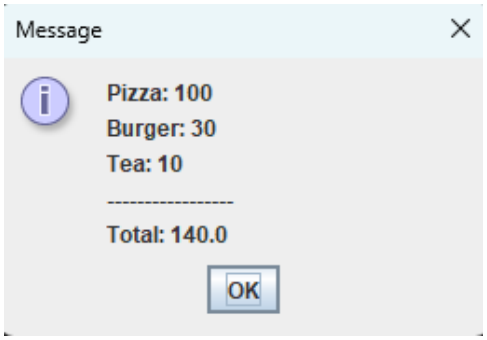
 # Source Code in JAVA:

```
import javax.swing.*;
 import java.awt.event.*;
public class BillGeneration extends JFrame implements ActionListener{
 JLabel l;
JCheckBox cb1,cb2,cb3;
 JButton b;
 BillGeneration(){
 l=new JLabel("Food Ordering System"); l.setBounds(50,50,300,20);
 cb1=new JCheckBox("Pizza @ 100"); cb1.setBounds(100,100,150,20);
 cb2=new JCheckBox("Burger @ 30"); cb2.setBounds(100,150,150,20);
 cb3=new JCheckBox("Tea @ 10"); cb3.setBounds(100,200,150,20);
 b=new JButton("Order");
b.setBounds(100,250,80,30);
 b.addActionListener(this); add(l);add(cb1);add(cb2);add(cb3);add(b);
setSize(400,400);
setLayout(null);
setVisible(true);
setDefaultCloseOperation(EXIT_ON_CLOSE);
}
public void actionPerformed(ActionEvent e){
float amount=0;
 String msg="";
 if(cb1.isSelected()){
 amount+=100;
 msg="Pizza: 100\n";
}
 if(cb2.isSelected()){
 amount+=30;
msg+="Burger: 30\n";
 }
 if(cb3.isSelected()){
```

```
 amount+=10;
msg+="Tea: 10\n";
}
msg+="----------------\n";
JOptionPane.showMessageDialog(this,msg+"Total: "+amount);
}
public static void main(String[] args) {
 new BillGeneration();
}
}
```

Output:





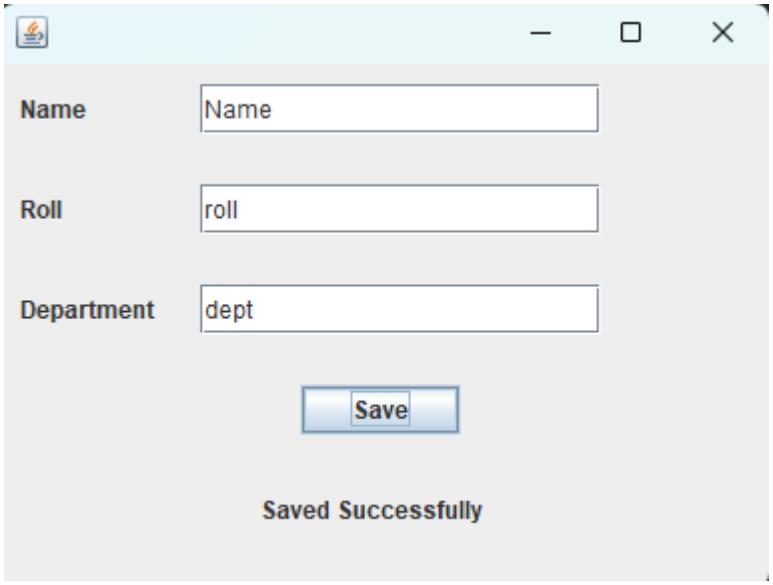
Lab-03: Write a JAVA Program to Create a Student form in GUI

 # Source Code in JAVA:

```
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JPanel;
import javax.swing.JTextField;
public class form implements ActionListener {
private static JLabel success;
private static JFrame frame;
private static JLabel label1,label2,label3;
private static JPanel panel;
private static JButton button;
private static JTextField userText1, userText2, userText3;
public static void main(String[] args) {
frame = new JFrame();
panel = new JPanel();
frame.setSize(400, 300);
frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
frame.add(panel);
panel.setLayout(null);
label1= new JLabel("Name");
label1.setBounds(10,10,80,25);
panel.add(label1);
  label2 = new JLabel("Roll");
  label2.setBounds(10,60,80,25);
  panel.add(label2);
  label3 = new JLabel("Department");
  label3.setBounds(10,110,80,25);
  panel.add(label3);
  userText1 = new JTextField("Enter Your Name");
```

```
 userText1.setBounds(100,10,200,25);
 panel.add(userText1);
 JTextField userText2 = new JTextField("Enter Your Roll");
 userText2.setBounds(100,60,200,25);
 panel.add(userText2);
 JTextField userText3 = new JTextField("Enter Your dept");
 userText3.setBounds(100,110,200,25);
 panel.add(userText3);
 button = new JButton("Save");
 button.setBounds(150, 160, 80, 25);
 button.addActionListener(new form());
 panel.add(button);
 success = new JLabel("");
 success.setBounds(130,210,300,25);
 panel.add(success);
 frame.setVisible(true); }
@Override
public void actionPerformed(ActionEvent e) {
 // TODO Auto-generated method stub
 success.setText("Saved Successfully");
}    }
```

Output:



Lab-04: Write a JAVA Program to develop a simple calculator in GUI

 # Source Code in JAVA:

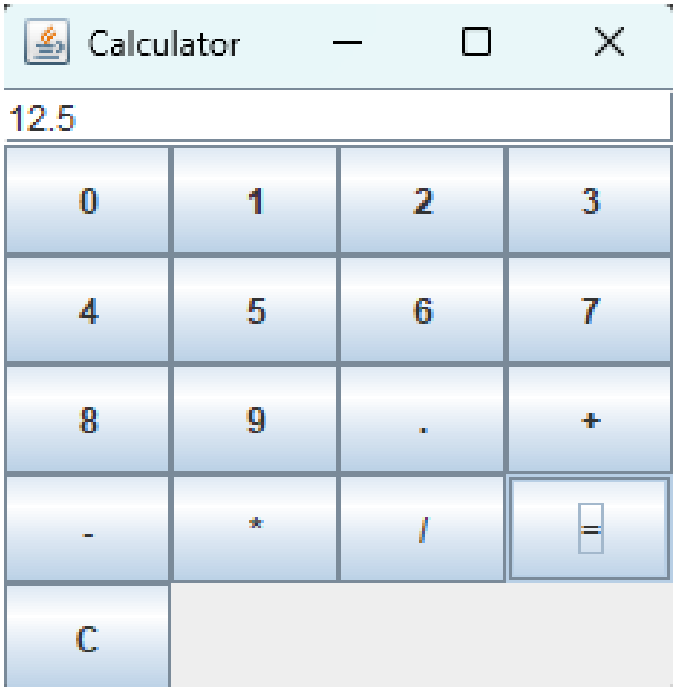
```
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import javax.swing.*;

public class calculator extends JFrame implements ActionListener {
  JButton b10,b11,b12,b13,b14,b15,b16;
  JButton b[]=new JButton[10];
  JTextField res;
  double n1=0,n2=0,r=0;
  char op;
  public calculator() {
    super("Calculator");
    setLayout(new BorderLayout());
    JPanel p=new JPanel();
    p.setLayout(new GridLayout(5,4));
    for(int i=0;i<=9;i++) {
      b[i]=new JButton(i+"");
      p.add(b[i]);
      b[i].addActionListener(this);  }
    b16=new JButton("."); // decimal point
    p.add(b16);
    b16.addActionListener(this);
    b10=new JButton("+");
    p.add(b10); b10.addActionListener(this);
    b11=new JButton("-");
    p.add(b11); b11.addActionListener(this);
    b12=new JButton("*");
    p.add(b12); b12.addActionListener(this);
    b13=new JButton("/");
    p.add(b13); b13.addActionListener(this);
```

```
    b14=new JButton("=");
    p.add(b14); b14.addActionListener(this);
    b15=new JButton("C");
    p.add(b15); b15.addActionListener(this);
    res=new JTextField(10);
    add(p,BorderLayout.CENTER);
    add(res,BorderLayout.NORTH);
    setVisible(true);
    setSize(250,250);   }
  public void actionPerformed(ActionEvent ae) {
    JButton pb=(JButton)ae.getSource();
    try {
      if(pb==b15) {
        r=n1=n2=0;
        res.setText("");    }
      else if(pb==b14) {
        n2 = Double.parseDouble(res.getText());
        eval();
        res.setText(String.valueOf(r));      }
      else {
        boolean opf=false;
        if(pb==b10){ op='+'; opf=true; }
        if(pb==b11){ op='-'; opf=true; }
        if(pb==b12){ op='*'; opf=true; }
        if(pb==b13){ op='/'; opf=true; }
        if(!opf) {
          for(int i=0;i<10;i++) {
            if(pb==b[i]) {
              res.setText(res.getText()+i);
            }   }
         if(pb==b16) {
            if(!res.getText().contains("."))
              res.setText(res.getText()+".");
          }     }
        else {
          n1 = Double.parseDouble(res.getText());
```

```
            res.setText(""); } }}
      catch(Exception e) {
        res.setText("Error");
      }  }
    void eval() {
      switch(op) {
        case '+': r=n1+n2; break;
        case '-': r=n1-n2; break;
        case '*': r=n1*n2; break;
        case '/':
          if(n2==0) throw new ArithmeticException();
          r=n1/n2; break;
      }   }
    public static void main(String arg[]) {
      new calculator();
    }
}
```

Output:



Lab-5 : Write a Java program to create thread class.

 # Source Code in JAVA:

```
class A extends Thread
{
   public void run()
   {
      for (int i = 1; i <= 5; i++)
      {
         System.out.println("From Thread A : i = " + i);
      }
      System.out.println("Exit from A");
   }
}

class B extends Thread
{
   public void run()
   {
      for (int j = 1; j <= 5; j++)
      {
         System.out.println("From Thread B : j = " + j);
      }
      System.out.println("Exit from B");
   }
}

class C extends Thread
{
   public void run()
   {
      for (int k = 1; k <= 5; k++)
      {
         System.out.println("From Thread C : k = " + k);
```

```
        }
        System.out.println("Exit from C");
    } }
class ThreadTest
{
    public static void main(String args[])
    {
        A objA = new A();
        B objB = new B();
        C objC = new C();
        objA.start();
        objB.start();
        objC.start();
    }
}
```

Output:

```
From Thread A : i = 1
From Thread B : j = 1
From Thread C : k = 1
From Thread A : i = 2
From Thread B : j = 2
From Thread C : k = 2
From Thread A : i = 3
From Thread C : k = 3
From Thread B : j = 3
From Thread A : i = 4
From Thread B : j = 4
From Thread C : k = 4
From Thread A : i = 5
Exit from A
From Thread B : j = 5
Exit from B
From Thread C : k = 5
Exit from C
```

Lab-06 : Write a Java program to illustrate yield(), stop() and sleep() method using thread.

 # Source Code in JAVA:

```
class A extends Thread {
   public void run() {
      for (int i = 1; i <= 5; i++) {
         if (i == 1)
            Thread.yield(); // Yield example
         System.out.println("From Thread A : i = " + i);
      }
      System.out.println("Exit from A");
   }}
class B extends Thread {
   public void run() {
      for (int j = 1; j <= 5; j++) {
         System.out.println("From Thread B : j = " + j);
         if (j == 3)
            this.stop(); // Stop example (deprecated)
      }
   }}
class C extends Thread {
   public void run() {
      for (int k = 1; k <= 5; k++) {
         System.out.println("From Thread C : k = " + k);
         try {
            Thread.sleep(1000); // Sleep example
         } catch (Exception e) {}
      }
      System.out.println("Exit from C");
   }}
class ThreadMethods {
   public static void main(String args[]) {
      A threadA = new A();
      B threadB = new B();
```

```
        C threadC = new C();
        System.out.println("Start thread A");
        threadA.start();
        System.out.println("Start thread B");
        threadB.start();
        System.out.println("Start thread C");
        threadC.start();
        System.out.println("End of main thread");
    }}
```

Output:

```
Start thread A
Start thread B
Start thread C
End of main thread

From Thread A : i = 1
From Thread A : i = 2
From Thread A : i = 3
From Thread A : i = 4
From Thread A : i = 5
Exit from A
From Thread B : j = 1
From Thread B : j = 2
From Thread B : j = 3
Exit from B

From Thread C : k = 1
From Thread C : k = 2
From Thread C : k = 3
From Thread C : k = 4
From Thread C : k = 5
Exit from C
```

Lab-07 : Write a Java program to use priority of thread.

 # Source Code in JAVA:

```
class A extends Thread {
   public void run() {
      System.out.println("threadA started");
      for (int i = 1; i <= 4; i++) {
         System.out.println("From Thread A : i = " + i);
      }
      System.out.println("Exit from A");
   }}
class B extends Thread {
   public void run() {
      System.out.println("threadB started");
      for (int j = 1; j <= 4; j++) {
         System.out.println("From Thread B : j = " + j);
      }
      System.out.println("Exit from B");
   }}
class C extends Thread {
   public void run() {
      System.out.println("threadC started");
      for (int k = 1; k <= 4; k++) {
         System.out.println("From Thread C : k = " + k);
      }
      System.out.println("Exit from C");
   }}
class ThreadPriority {
   public static void main(String args[]) {
      A threadA = new A();
      B threadB = new B();
      C threadC = new C();
      // Thread priorities
      threadC.setPriority(Thread.MAX_PRIORITY);       // 10
      threadB.setPriority(threadA.getPriority() + 1); // 6
```

```
  threadA.setPriority(Thread.MIN_PRIORITY);        // 1
        // Start threads
        System.out.println("Start thread A");
        threadA.start();
        System.out.println("Start thread B");
        threadB.start();
        System.out.println("Start thread C");
        threadC.start();

        System.out.println("End of main thread");
    }
}
```

Output:

```
Start thread A
Start thread B
Start thread C
End of main thread
threadA started
From Thread A : i = 1
From Thread B : j = 1
threadC started
From Thread C : k = 1
From Thread A : i = 2
From Thread B : j = 2
From Thread C : k = 2
From Thread A : i = 3
From Thread B : j = 3
From Thread C : k = 3
From Thread A : i = 4
From Thread B : j = 4
From Thread C : k = 4
Exit from A
Exit from B
Exit from C
```

Lab-08:Write a Java program to illustrate suspend, resume and stop operation of thread.

 # Source Code in JAVA:

```
class SusResStop implements Runnable {
   Thread th;
   boolean suspend_flag, stop_flag;
   SusResStop(String name) {
      th = new Thread(this, name);
      suspend_flag = false;
      stop_flag = false;
      th.start(); }
   public void run() {
      try {
         int j = 1;
         while (j <= 20) {
            synchronized (this) {
               while (suspend_flag)
                  wait();

               if (stop_flag)
                  break;  }
            System.out.println(th.getName() + " is running: " + j++);
            Thread.sleep(500); // just to slow down output
         }   }
catch (InterruptedException e) {
         System.out.println("Thread interrupted");
      }}
   synchronized void my_suspend() {
      suspend_flag = true;    }
   synchronized void my_resume() {
      suspend_flag = false;
   notify();   }
   synchronized void my_stop() {
      suspend_flag = false;
```

```
        stop_flag = true;
        notify();
    }}
public class EgSRS {
    public static void main(String args[]) {
        try {
            System.out.println("started");
            SusResStop S_R_S_T = new SusResStop("SRS");
            System.out.println("Thread S_R_S_T is created and started");
            Thread.sleep(2000);
            S_R_S_T.my_suspend();
            System.out.println("Thread S_R_S_T is suspended");
            Thread.sleep(2000);
            S_R_S_T.my_resume();
            System.out.println("Thread S_R_S_T is resumed");
            Thread.sleep(2000);
            S_R_S_T.my_stop();
            System.out.println("Thread S_R_S_T is stopped");
        } catch (InterruptedException e) {
            System.out.println("Generated interrupted exception");
        }   }}
```

Output:

```
started
Thread S_R_S_T is created and started
SRS is running: 1
SRS is running: 2
SRS is running: 3
SRS is running: 4
Thread S_R_S_T is suspended
Thread S_R_S_T is resumed
SRS is running: 5
SRS is running: 6
SRS is running: 7
SRS is running: 8
Thread S_R_S_T is stopped
```

Lab-09: Write a client-server TCP socket program in java that the server listens for connection requests, and whatever message the client sends, the server converts it to uppercase and sends it back.

 # Source Code in JAVA:

Server.java:

```
import java.io.*;
import java.net.*;

public class Server {
    public static void main(String[] args) {
        int port = 5000; // Server port
        try (ServerSocket serverSocket = new ServerSocket(port)) {
            System.out.println("Server started. Listening on port " + port);

            while (true) {
                Socket clientSocket = serverSocket.accept();
                System.out.println("Client connected: " +
clientSocket.getInetAddress());
                new Thread(() -> handleClient(clientSocket)).start();
            }

        } catch (IOException e) {
            e.printStackTrace();
        }
    }

    private static void handleClient(Socket clientSocket) {
        try (
            BufferedReader in = new BufferedReader(new
InputStreamReader(clientSocket.getInputStream()));
            PrintWriter out = new
PrintWriter(clientSocket.getOutputStream(), true)
        ) {
```

```
            String message;
            while ((message = in.readLine()) != null) {
                System.out.println("Received: " + message);
                String upperCaseMessage = message.toUpperCase();
                out.println(upperCaseMessage); // send back to client
            }
        } catch (IOException e) {
            e.printStackTrace();
        } finally {
            try {
                clientSocket.close();
            } catch (IOException e) {
                e.printStackTrace();
            }
        }
    }
}
```

Client.java:

```
import java.io.*;
import java.net.*;
import java.util.Scanner;

public class Client {
    public static void main(String[] args) {
        String host = "localhost"; // Server address
        int port = 5000;           // Server port
        try (Socket socket = new Socket(host, port);
            BufferedReader in = new BufferedReader(new
InputStreamReader(socket.getInputStream()));
            PrintWriter out = new PrintWriter(socket.getOutputStream(),
true);
            Scanner scanner = new Scanner(System.in)) {

            System.out.println("Connected to server at " + host + ":" + port);
```

```
            while (true) {
                System.out.print("Enter message (or 'exit' to quit): ");
                String message = scanner.nextLine();
                if (message.equalsIgnoreCase("exit")) break;
                out.println(message);          // send to server
                String response = in.readLine(); // get response
                System.out.println("Server response: " + response);
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

Output:

Server Terminal Output

```
Server started. Listening on port 5000
Client connected: /127.0.0.1
Received: hello server
Received: how are you
Received: this is a test
Client disconnected.
```

Client Terminal Output

```
Connected to server at localhost:5000
Enter message (or 'exit' to quit): hello server
Server response: HELLO SERVER
Enter message (or 'exit' to quit): how are you
Server response: HOW ARE YOU
Enter message (or 'exit' to quit): this is a test
Server response: THIS IS A TEST
Enter message (or 'exit' to quit): exit
Connection closed.
```

Lab-10: Write a client-server UDP socket program in java that the server listens for connection requests, and whatever message within 1024 bytes the client sends, the server converts it to uppercase and sends it back after 6 ms.

 # Source Code in JAVA:

UDPServer.java:

```
import java.net.DatagramPacket;
import java.net.DatagramSocket;
import java.net.InetAddress;

public class UDPServer {
   public static void main(String[] args) {
      int port = 9876; // server port
      try (DatagramSocket serverSocket = new DatagramSocket(port)) {
         System.out.println("Server is running on port " + port);

         byte[] receiveBuffer = new byte[1024];
         byte[] sendBuffer;

         while (true) {
            // Receive packet
            DatagramPacket receivePacket = new
DatagramPacket(receiveBuffer, receiveBuffer.length);
            serverSocket.receive(receivePacket);

            // Extract message
            String message = new String(receivePacket.getData(), 0,
receivePacket.getLength());
            System.out.println("Received: " + message);

            // Convert to uppercase after 6 ms delay
            Thread.sleep(6);
            String upperMessage = message.toUpperCase();
```

```
            sendBuffer = upperMessage.getBytes();

            // Send back to client
            InetAddress clientAddress = receivePacket.getAddress();
            int clientPort = receivePacket.getPort();
            DatagramPacket sendPacket = new
DatagramPacket(sendBuffer, sendBuffer.length, clientAddress,
clientPort);
            serverSocket.send(sendPacket);

            System.out.println("Sent back: " + upperMessage);
        }
    } catch (Exception e) {
        e.printStackTrace();
    }
  }
}
```

<u>UDPClient.java:</u>

```
import java.net.DatagramPacket;
import java.net.DatagramSocket;
import java.net.InetAddress;
import java.util.Scanner;

public class UDPClient {
   public static void main(String[] args) {
      String serverHost = "localhost"; // or server IP
      int serverPort = 9876;

      try (DatagramSocket clientSocket = new DatagramSocket();
          Scanner sc = new Scanner(System.in)) {

         System.out.print("Enter message to send: ");
         String message = sc.nextLine();
         byte[] sendBuffer = message.getBytes();
```

```
        InetAddress serverAddress =
InetAddress.getByName(serverHost);
        DatagramPacket sendPacket = new DatagramPacket(sendBuffer,
sendBuffer.length, serverAddress, serverPort);
        clientSocket.send(sendPacket);

        // Receive response
        byte[] receiveBuffer = new byte[1024];
        DatagramPacket receivePacket = new
DatagramPacket(receiveBuffer, receiveBuffer.length);
        clientSocket.receive(receivePacket);

        String response = new String(receivePacket.getData(), 0,
receivePacket.getLength());
        System.out.println("Response from server: " + response);

    } catch (Exception e) {
        e.printStackTrace();
    }
  }
}
```

output:

Server Side:
Server is running on port 9876
Received: hello udp server
Sent back: HELLO UDP SERVER

Client Side:
Enter message to send: hello udp server
Response from server: HELLO UDP SERVER

Lab-11: Suppose we have an MS Access database named ICE_PUST which has a table named by Student with field (Name, Email, and Phone). Using this database answer the following questions:

a. Write down a java program to insert data into the Student table of the ICE_PUST database.

b. Create a java program to print all student records from the Student table of the ICE_PUST database.

 # Source Code in JAVA:

Create MS Access database:

```
import java.sql.*;
import java.io.File;

public class CreateDB {
    public static void main(String[] args) {
        try {
            // 1 Folder path
            String folderPath = "C:/Databases";
            File folder = new File(folderPath);
            if (!folder.exists()) {
                folder.mkdirs(); // Folder create
                System.out.println("Folder created: " + folderPath);
            }

            // 2 Database path
            String dbPath = folderPath + "/ICE_PUST.accdb";

            // 3 Load UCanAccess Driver
            Class.forName("net.ucanaccess.jdbc.UcanaccessDriver");

            // 4 Connect to database (auto-create if not exists)
```

```
        Connection con =
DriverManager.getConnection("jdbc:ucanaccess://" + dbPath);

        Statement st = con.createStatement();

        // 5 Create table if not exists
        try {
            st.executeUpdate(
                "CREATE TABLE Student (" +
                "Name TEXT, " +
                "Email TEXT, " +
                "Phone TEXT)"
            );
            System.out.println("Table 'Student' created successfully!");
        } catch (SQLException e) {
            System.out.println("Table already exists. Skipping creation.");
        }

        con.close();
        System.out.println("Database ready at: " + dbPath);

    } catch (Exception e) {
        e.printStackTrace();
    }
  }
}
// javac -cp ".;ucanaccess-5.0.1.jar;lib/*" CreateDB.java
//java -cp ".;ucanaccess-5.0.1.jar;lib/*" CreateDB
```

a. Java Program to Insert Data into Student Table

```
import java.sql.*;
import java.util.Scanner;

public class InsertStudent {
  public static void main(String[] args) {
```

```
        String dbURL =
"jdbc:ucanaccess://C:/Databases/ICE_PUST.accdb";

        try {
            Class.forName("net.ucanaccess.jdbc.UcanaccessDriver");

            Connection conn = DriverManager.getConnection(dbURL);
            Scanner sc = new Scanner(System.in);

            System.out.print("Enter student name: ");
            String name = sc.nextLine();

            System.out.print("Enter student email: ");
            String email = sc.nextLine();

            System.out.print("Enter student phone: ");
            String phone = sc.nextLine();

            String sql = "INSERT INTO Student (Name, Email, Phone)
VALUES (?, ?, ?)";
            PreparedStatement pst = conn.prepareStatement(sql);

            pst.setString(1, name);
            pst.setString(2, email);
            pst.setString(3, phone);

            int rowsInserted = pst.executeUpdate();
            if (rowsInserted > 0) {
                System.out.println("Student inserted successfully!");
            }

            conn.close();
        } catch (Exception e) {
            e.printStackTrace();
        }
    }}
```

```
//javac -cp ".;ucanaccess-5.0.1.jar;lib/*" InsertStudent.java
//java -cp ".;ucanaccess-5.0.1.jar;lib/*" InsertStudent
```

b. Java Program to Print All Student Records

```
import java.sql.*;

public class DisplayStudents {
   public static void main(String[] args) {

      String dbURL =
"jdbc:ucanaccess://C:/Databases/ICE_PUST.accdb";

      try {
         Class.forName("net.ucanaccess.jdbc.UcanaccessDriver");

         Connection conn = DriverManager.getConnection(dbURL);

         String sql = "SELECT * FROM Student";

         Statement stmt = conn.createStatement();
         ResultSet rs = stmt.executeQuery(sql);

         System.out.println("Student Records:");
         System.out.println("-----------------------------");

         while (rs.next()) {
            String name = rs.getString("Name");
            String email = rs.getString("Email");
            String phone = rs.getString("Phone");

            System.out.println("Name: " + name);
            System.out.println("Email: " + email);
            System.out.println("Phone: " + phone);
            System.out.println("-----------------------------");
         }
```

```
            conn.close();

        } catch (Exception e) {
            System.out.println(e);
        }
    }
}
//javac -cp ".;ucanaccess-5.0.1.jar;lib/*" DisplayStudents.java
//java -cp ".;ucanaccess-5.0.1.jar;lib/*" DisplayStudents
```

output:

InsertStudent.java Program Output (Sample):
Enter student name: Rahim Khan
Enter student email: rahim.khan@mail.com
Enter student phone: 01712345678
A new student was inserted successfully!

Enter student name: Fatima Begum
Enter student email: fatima.begum@mail.com
Enter student phone: 01898765432
A new student was inserted successfully!

DisplayStudents.java Program Output:
Student Records:

Name: Rahim Khan
Email: rahim.khan@mail.com
Phone: 01712345678

Name: Fatima Begum
Email: fatima.begum@mail.com
Phone: 01898765432

Lab-12: Write a Java program to perform insert, delete, edit and show data using MySQL database.

 # Source Code in JAVA:

```
import java.sql.*;
import java.util.Scanner;

public class StudentLab {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      try {
         // Load SQL Server JDBC Driver
         Class.forName("com.microsoft.sqlserver.jdbc.SQLServerDriver
");
         // Connection details - CHANGE username & password
  String url =
"jdbc:sqlserver://localhost:1433;databaseName=ICE_PUST;encrypt=tru
e;trustServerCertificate=true;integratedSecurity=true";

         Connection conn = DriverManager.getConnection(url);

         while (true) {
            System.out.println("\n1.Insert 2.Delete 3.Edit 4.Show
5.Exit");
            System.out.print("Choice: ");
            int ch = sc.nextInt(); sc.nextLine();

            if (ch == 1) { // Insert
               System.out.print("Name: "); String n = sc.nextLine();
               System.out.print("Age: "); int a = sc.nextInt();
sc.nextLine();
               System.out.print("Email: "); String e = sc.nextLine();

               PreparedStatement p = conn.prepareStatement(
```

```
                "INSERT INTO students(name, age, email) VALUES (?,
?, ?)"
            );
            p.setString(1, n);
            p.setInt(2, a);
            p.setString(3, e);
            p.executeUpdate();
            System.out.println("Inserted!");
        }
        else if (ch == 2) { // Delete
            System.out.print("ID to delete: "); int id = sc.nextInt();
sc.nextLine();
            PreparedStatement p = conn.prepareStatement(
                "DELETE FROM students WHERE id = ?"
            );
            p.setInt(1, id);
            int r = p.executeUpdate();
            System.out.println(r > 0 ? "Deleted!" : "ID not found!");
        }
        else if (ch == 3) { // Edit
            System.out.print("ID to edit: "); int id = sc.nextInt();
sc.nextLine();
            System.out.print("New Name: "); String n = sc.nextLine();
            System.out.print("New Age: "); int a = sc.nextInt();
sc.nextLine();
            System.out.print("New Email: "); String e = sc.nextLine();

            PreparedStatement p = conn.prepareStatement(
                "UPDATE students SET name = ?, age = ?, email = ?
WHERE id = ?"
            );
            p.setString(1, n);
            p.setInt(2, a);
            p.setString(3, e);
            p.setInt(4, id);
            int r = p.executeUpdate();
```

```
                System.out.println(r > 0 ? "Updated!" : "ID not found!");
            }
            else if (ch == 4) { // Show
                Statement s = conn.createStatement();
                ResultSet rs = s.executeQuery("SELECT * FROM
students");
                System.out.println("ID\tName\tAge\tEmail");
                while(rs.next()) {
                    System.out.println(
                        rs.getInt("id") + "\t" +
                        rs.getString("name") + "\t" +
                        rs.getInt("age") + "\t" +
                        rs.getString("email")
                    );
                }}
            else break; // Exit   }
        conn.close(); }
    catch(Exception ex){
        System.out.println("Error: " + ex.getMessage());
        ex.printStackTrace();
    }
    sc.close();
  }}
// javac -cp ".;jars\mssql-jdbc-13.4.0.jre11.jar" StudentLab.java
//java -cp ".;jars\mssql-jdbc-13.4.0.jre11.jar" StudentLab
```

MySQL Table Structure:

```
CREATE DATABASE ICE_PUST;
USE ICE_PUST;
CREATE TABLE students (
  id INT PRIMARY KEY IDENTITY(1,1),
  name VARCHAR(50),
  age INT,
  email VARCHAR(50)
);
```

Sample Output:

```
1.Insert 2.Delete 3.Edit 4.Show 5.Exit
Choice: 1
Name: John
Age: 21
Email: john@example.com
Inserted!

1.Insert 2.Delete 3.Edit 4.Show 5.Exit
Choice: 4
ID      Name    Age     Email
1       John    21      john@example.com

1.Insert 2.Delete 3.Edit 4.Show 5.Exit
Choice: 3
ID to edit: 1
New Name: John Doe
New Age: 22
New Email: john.doe@example.com
Updated!

1.Insert 2.Delete 3.Edit 4.Show 5.Exit
Choice: 4
ID      Name        Age     Email
1       John Doe    22      john.doe@example.com

1.Insert 2.Delete 3.Edit 4.Show 5.Exit
Choice: 2
ID to delete: 1
Deleted!

1.Insert 2.Delete 3.Edit 4.Show 5.Exit
Choice: 4
No records found.
```